



UNIVERSITÀ DI PARMA

Complessità computazionale Ricerca & Ordinamento

*Sorting something that you will never search
is a complete waste; searching something you
never sorted is merely inefficient."*

Brian Christian



UNIVERSITÀ DI PARMA

Complessità Computazionale

Teoria della Complessità Computazionale

- La teoria della complessità è un tentativo di dare una risposta matematica a domande come:
 - Cosa vuol dire che un algoritmo è più efficiente di un altro?
 - In che senso un problema di calcolo è più difficile di un altro?
 - Che cosa rende un problema “affrontabile”?
 - Esistono soluzioni ottimali per problemi di calcolo effettivamente risolvibili?
 - Quali problemi si possono risolvere meglio parallelizzando?
 - ecc.

A parità di problema

- Uno stesso problema può essere risolto usando approcci differenti
- Algoritmi diversi possono avere diversa efficienza
- Dove per “efficienza” posso scegliere parametri differenti
 - Tempo di esecuzione
 - Occupazione memoria
 - ...

- Limitiamoci a questo → discorso comunque facilmente estendibile
- Come lo misuro?
- Pratica: cronometro
- Teoria: stima in base al “numero di passi”

Possibile misura della complessità

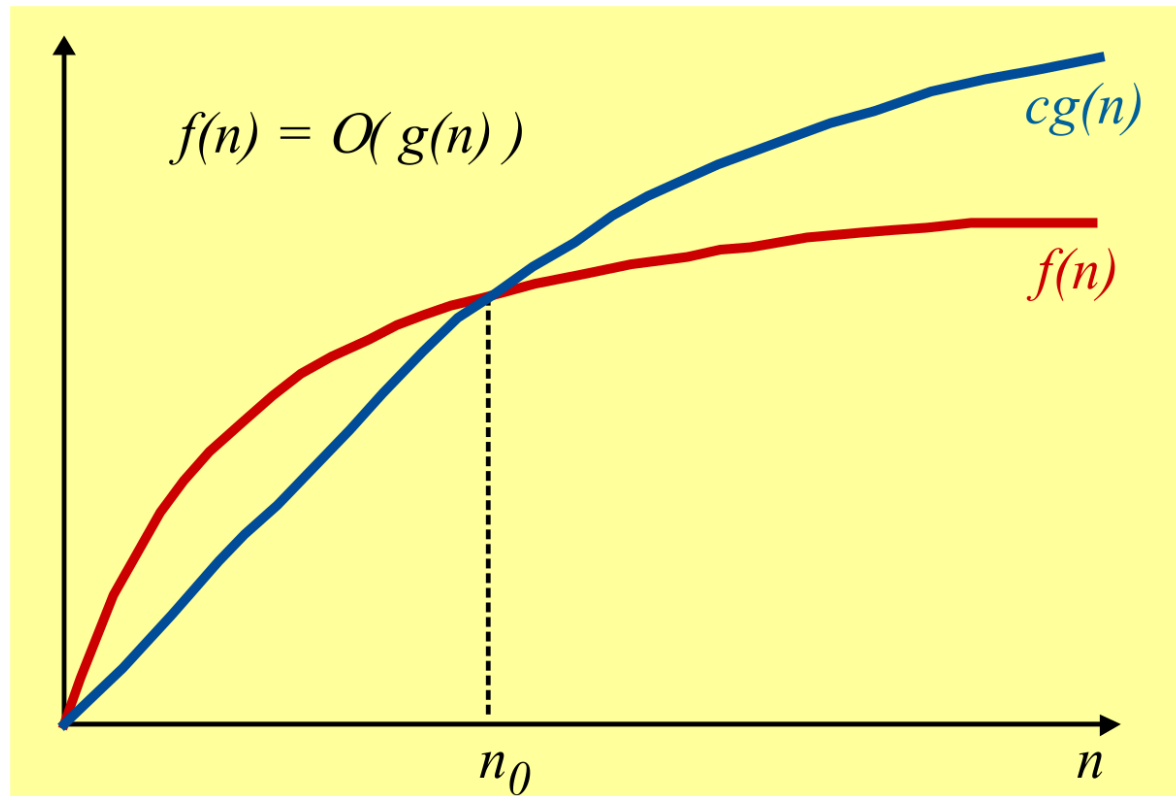
- Algoritmo $\rightarrow$ sequenza finita di passi di risoluzione
- Numero passi $\rightarrow$ misura efficienza di un algoritmo
 - Spesso dipendente da mole dati $\rightarrow n$
 - Quindi **numero di passi** sarà **$f(n)$**
- Non ci interessa per n piccoli

- Numero di passi $f(n)$ dove n “grandezza” input
- Una funzione $f(n)$ è di ordine $g(n)$ quando:

$$\exists n_0 \geq 0 \text{ e } c > 0 \text{ tali che } f(n) \leq c g(n) \quad \forall n > n_0$$

- In tal caso si scrive che $f(n) = O(g(n))$

Notazione asintotica O grande



$\exists n_0 \geq 0$ e $c > 0$ tali che $f(n) \leq g(n) \quad \forall n > n_0$

- Esempi

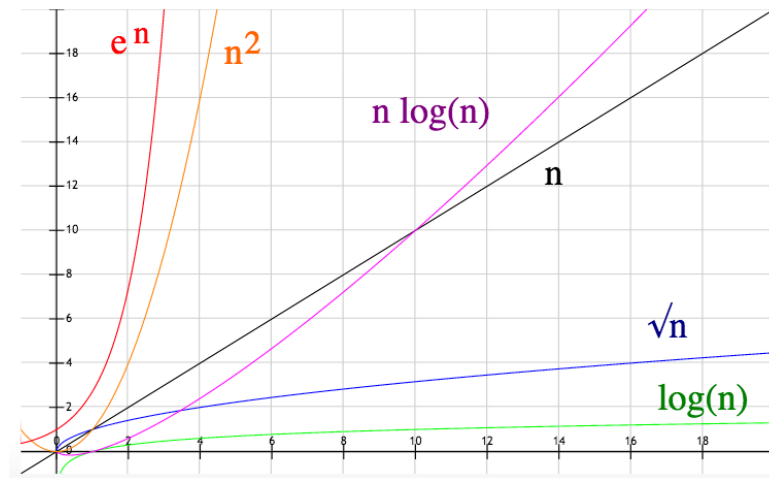
- $f(n) = 4n + 18 \rightarrow O(n)$

- $f(n) = 20n^2 + 3n + 7 \rightarrow O(n^2)$

- $f(n) = n \log(n) + n^2 \rightarrow O(n^2)$

Notazione asintotica O grande

- $O(1)$ → Complessità costante
- $O(\log n)$ → Complessità logaritmica
- $O(n)$ → Complessità lineare
- $O(n \cdot \log n)$ → Complessità pseudolineare
- $O(n^2)$ → Complessità quadratica
- $O(n^k)$ → Complessità polinomiale
- $O(a^n)$ → Complessità esponenziale



- C'è una costante moltiplicativa
 - Sia n che $10^{18}n$ sono $O(n)$
- Per n “non troppo grandi” non ci dice nulla
- Parlando di tempi esecuzione:
 - Non tutti i passi hanno lo stesso “peso”
 - Gerarchie di memorie a velocità differenti

Pratica della complessità computazionale

- Non esiste l'algoritmo „migliore“
 - Complessità computazionale
 - Tempi computazionali (corso di ingegneria)
 - Efficienza dell'implementazione
 - Quantità di memoria usata
 - Realizzabilità



UNIVERSITÀ DI PARMA

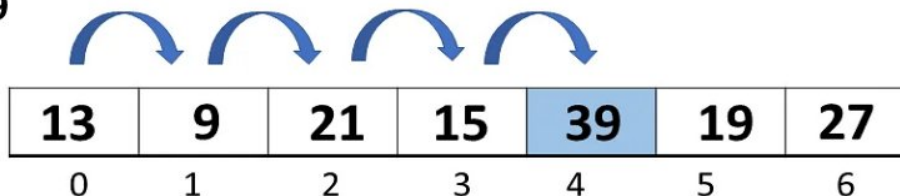
Algoritmi di Ricerca

- Cosa s'intende per algoritmo di ricerca?
 - Metodo per trovare uno specifico dato all'interno di una grande mole di dati
- Per noi “grande mole di dati” $\rightarrow$ array
 - Ma discorso generale
- Due tipi possibili di ricerca

- “Scorro” l'array elemento per elemento
 - Per ogni elemento confronto con quanto cerco
 - Se lo trovo → termino
 - Se arrivo a fine array → termino

cerca elemento

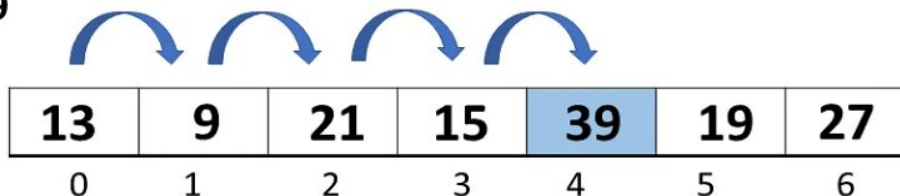
39



- Efficienza:
 - Molto semplice da implementare
 - Se $n \rightarrow$ numero elementi array
 - Non lo trovo $\rightarrow$ scandisco n elementi
 - Se lo trovo $\rightarrow$ mediamente scandisco $\frac{1}{2}n$ elementi
 - $O(n)$
- Unica scelta possibile per array non ordinati

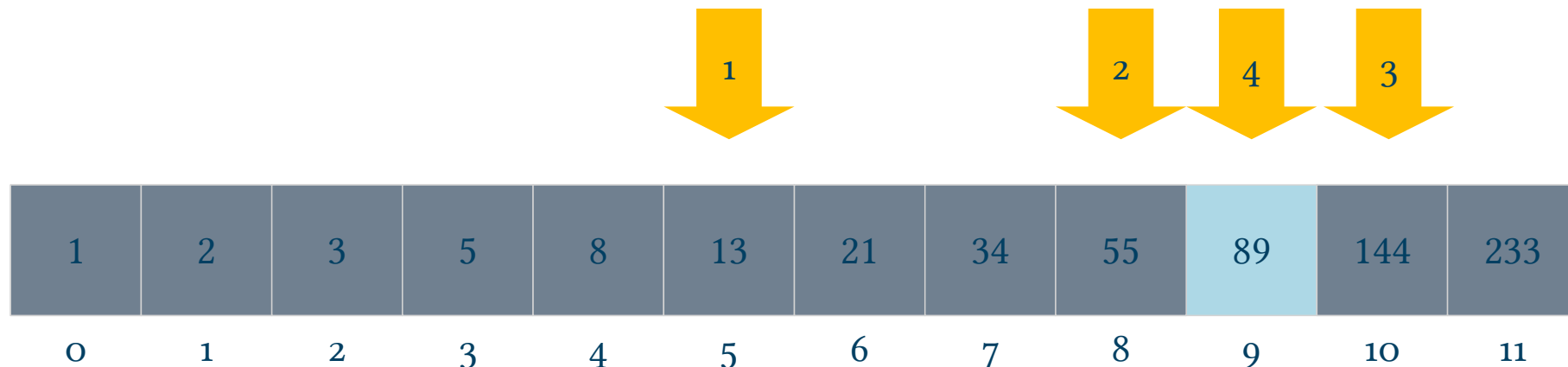
cerca elemento

39



- Possibile se e solo se array ordinato
- Approccio stile simil-“dizionario”
- Parto da elemento di mezzo e confronto con ciò che cerco
 - Se uguale $\rightarrow$ termino (l'ho trovato!)
 - Se minore $\rightarrow$ ripeto per semi-array “inferiore”
 - Se maggiore $\rightarrow$ ripeto per semi-array “superiore”
 - Termino se arrivo ad array di singolo elemento
- Vedremo successivamente funzione C `bsearch()`

- Molto efficiente $O(\log_2 n)$
- Esempio: cerchiamo se nel seguente array c'è 89 (e dove...)



$89 > 13$!

Trovato in 8 passi!



UNIVERSITÀ DI PARMA

Algoritmi di Ordinamento

- Cosa intendiamo per ordinamento?
 - L'obiettivo è rimescolare “dati” ordinandoli in base a regole (alfabetico? numerico?)
 - Chiave di ordinamento: elemento usato per l'ordinamento
 - Esempio: nome e cognome → ordino per cognome



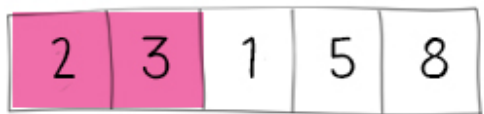
- Idea: spostare gli elementi “piú grandi” verso l’“alto”
- Scorro piú volte gli elementi, ad ogni passo confronto elementi contigui ed eventualmente li scambio

Bubblesort

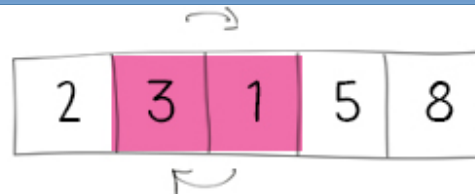
1. Start from the beginning and compare adjacent elements. Swap them if needed



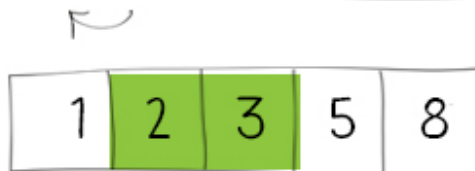
round 2



3. we can stop comparisons for elements already in the right position



round 3



round 4



4. following passes are then shorter

2. at the end of the first round the “largest” element is in the final position
Start another “pass”

- Little optimization:
 - no swaps in a round $\rightarrow$ terminate
- **Execution time complexity**
 - **Best case:** already ordered data, n comparisons $\rightarrow O(n)$
 - **Worst/Average case:** $\sim n(n-1)/2$ operations $\rightarrow O(n^2)$
- **Memory:** only a little additional space is needed $\rightarrow O(1)$
- Not efficient, simple implementation, but can outperform most complex algorithm when almost sorted data

- Creato nel 1960 da Hoare
- ampiamente utilizzato
 - veloce
 - relativamente facile da implementare grazie a ricorsione

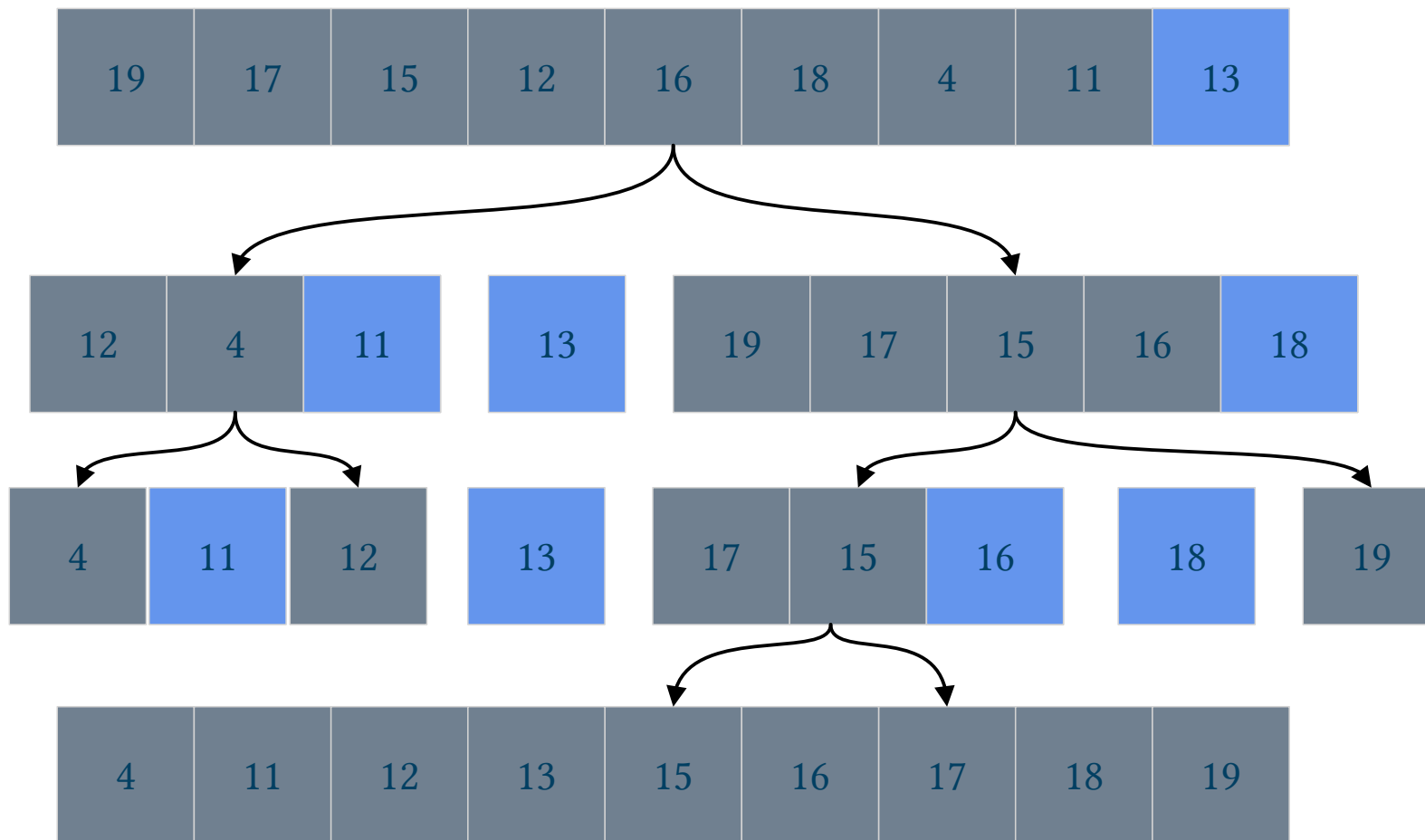
- Algoritmo divide-et-impera
 - si sceglie un elemento dell'insieme da ordinare: **pivot** (o perno)
 - **partizionamento**: si suddividono gli elementi in due sottoinsiemi:
 - elementi con chiave maggiore a quella del pivot
 - elementi con chiave minore di quella del pivot
 - si ripete la procedura sui due sottoinsiemi
- Facile implementazione ricorsiva

if $n \leq 1$ termina

altrimenti

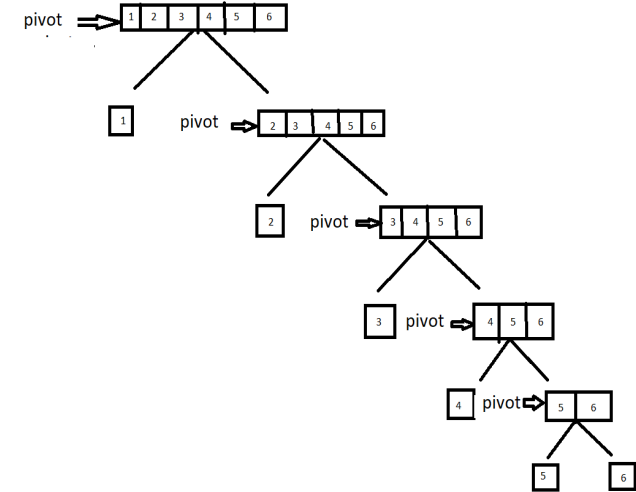
$Q(n) =$ scegli pivot e suddividi insieme
in elementi $\leq$ e $>$ pivot di numerosità a e b
con $a, b < n$
 $Q(a) + Q(b)$

Quicksort



1. Select a pivot
2. Partition data
3. Pivot already in right position
4. Again on subsets!
5. until single elements

- Execution time complexity
 - Best/Average case:
 - Every time I divide the set by 2 $\rightarrow \log_2(n)$
 - And compare the pivot against other elements $\rightarrow n$
 - $O(n \cdot \log_2(n))$
 - Worst case: already ordered set
 - Dividing by 2 is not so efficient, all elements on a single side $\rightarrow n$
 - Comparison as before $\rightarrow n$
 - $O(n^2)$



- Space complexity
 - Single array
 - Additional space on the call stack
 - $O(\log_2(n))$ - $O(n)$

Summary



Algorithm	Time Complexity			Auxiliary space
	Best	Average	Worst	Worst
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell sort	$O(n \cdot \log_2(n))$	$O(n \cdot \log_2(n))$	$O(n^{3/2})$	$O(1)$
Quick sort	$O(n \cdot \log_2(n))$	$O(n \cdot \log_2(n))$	$O(n^2)$	$O(n)$
Merge sort	$O(n \cdot \log_2(n))$	$O(n \cdot \log_2(n))$	$O(n \cdot \log_2(n))$	$O(n)$
Tim sort	$O(n)$	$O(n \cdot \log_2(n))$	$O(n \cdot \log_2(n))$	$O(n)$